شرح المحاضرة 01: ليه الكورس ده وليه Python (Why This Course, and Why Python)

المادة: DSA27 — هياكل البيانات والخوارزميات (Data Structures and Algorithms)
الدكتور: Dr. Haitham A. El-Ghareeb
الأسبوع: 1 · الترم الأول (22 September 2026)

فهرس المحتوى

1. المصادر وإزاي تجهّز جهازك 2
2. هل كل لغات البرمجة زي بعض؟ 3
3. القرارات ليها أصحاب 4
4. لغة C: منين جت وليه مهمة 6
5. تصحيح من الدكتور (Knuth) 7
6. البرادايمز: نفس الشغلانة بتلات طرق 8
7. أنظمة الأنواع: محورين مش محور واحد 10
8. الذاكرة: مين بينضّف؟ 12
9. نقارن اللغات بأمانة 13
10. ليه Python في الكورس ده 15
11. من المتغير لهيكل البيانات 17
12. التكلفة هي الموضوع كله 20
13. الكورس نفسه رسميًا 24
14. الملخص: ست حاجات تحفظها 27

1) المصادر وإزاي تجهّز جهازك

المصادر بتاعت الكورس

إيه	فين
الكود والتمارين والملاحظات	https://github.com/helghareeb/DSA27
أخبار الكورس والإعلانات	WhatsApp channel
بعض (مش كل) المحاضرات فيديو	https://www.youtube.com/@0xHGH

- **الإعلانات** بتنزل على قناة الـ WhatsApp الأول.
- **الـ repository هو المصدر الحقيقي** لأي حاجة تقنية: لو القناة والـ repository اتخالفوا، يبقى الـ repository هو الصح والدكتور غلط في القناة.
- الـ QR code بتاع القناة موجود في `docs/assets/whatsapp-channel-qr.jpg` وفي صفحة اللينكات `docs/links.md`.

أول حاجة تعملها النهارده

```
git clone https://github.com/helghareeb/DSA27.git
cd DSA27
py -3.13 -m venv .venv
.\.venv\Scripts\activate
pip install -r requirements.txt
pytest -m "not challenge"
```

الخطوات بالترتيب:

1. `git clone` — تنزّل نسخة من الـ repository على جهازك.
2. `cd DSA27` — تدخل الفولدر.
3. `py -3.13 -m venv .venv` — تعمل **virtual environment** (بيئة معزولة) بـ Python 3.13.
4. `activate` — تشغّل البيئة دي.
5. `pip install -r requirements.txt` — تنزّل المكتبات المطلوبة جوه البيئة.
6. `pytest -m "not challenge"` — تشغّل الاختبارات اللي **مش** تمارين.

> **نقطة مهمة:** كل الاختبارات دي لازم تنجح **قبل** ما تكتب سطر واحد من كود الكورس.

الاختبارات دي **مش بتختبر الخوارزميات بتاعتك**، دي بتختبر **جهازك**:

- نسخة الـ interpreter.
- إنك جوه virtual environment.

- مكتبات الرسم (plotting stack).
- برنامج Graphviz (الـ `dot` binary).
- دوال الرسم المساعدة في فولدر `viz/`.

لو واحد فشل، الرسالة بتقول بالظبط إيه الناقص. صلّحه الأول، لأن إنك تـ debug linked list على بيئة مكسورة ده تضييع ليلة.

> **انتبه:** على Windows فيه فخ مشهور: بتكتب Python فيفتح لك Microsoft Store بدل ما يشغّل Python. الحل في `README.md` بتاع الـ repository.

---

2) هل كل لغات البرمجة زي بعض؟

الحجة اللي بتبان صح

- أي لغة تقدر تطبع `Hello, World`.
- أي لغة تقدر تجمع رقمين.
- أي لغة في الآخر بتتحول لتعليمات نفس المعالج (processor) بينفذها.

يبقى كل لغات البرمجة زي بعض، والفرق في الـ syntax بس.

الدكتور بيقول إن دي أكتر حاجة طالب سنة تانية ذكي بيصدقها، وهيقعد عليها وقت لأن باقي الكورس مبني على إننا نفككها.

الحجة مش غبية — فيها مقدمة صحيحة

من ناحية **إيه اللي ممكن يتحسب** (what can be computed)، اللغات اللي هتقابلها **متساوية**: كلها **Turing-complete**. يعني أي برنامج تكتبه بلغة تقدر تكتبه بلغة تانية. دي **نظرية حقيقية** ومهم تعرفها.

الغلط فين؟

الغلط في كلمة **"زي بعض"**. التساوي في Turing بيقولك إن **مجموعة المسائل اللي تتحل واحدة**، لكن **مبيقولش حاجة** عن:

- **تكلفة الكتابة**.
- تكلفة **القراءة بعد 6 شهور**.
- تكلفة **التشغيل**.
- أنهي أخطاء اللغة بتسيبك تعملها **بصمت**.
- كام واحد يقدر **يصيّن** الكود (maintain).

يعني Turing-equivalence معناها: **نفس المسائل قابلة للحل**. لكن مش معناها إنها تتحل:

- بنفس **التكلفة**.

- بنفس **عدد الناس**.
- في نفس **الوقت**.
- بنفس **عدد الأخطاء اللي بتوصل للـ production**.

> **للامتحان:** لو اتسألت "هل كل اللغات متساوية؟" الإجابة: **متساوية في القدرة الحسابية (Turing-complete) لكن مختلفة في التكلفة** (كتابة، قراءة، تشغيل، أخطاء، صيانة).

اللغة = مجموعة قرارات

اللغة مش **أنبوبة محايدة** بتصب فيها المنطق. اللغة هي **مجموعة قرارات** عن:

- إيه السهل وإيه الصعب.
- إيه الممنوع.
- إيه اللي **يفشل بصوت عالي** (loud) وإيه اللي **يفشل بصمت** (silent).

القرارات دي **اتاخدت بواسطة ناس**، لأسباب، والأسباب غالبًا **مكتوبة**.

السلسلة اللي رسمها الدكتور على السبورة:

Programming Language → Design → Philosophy

- اقراها **من اليمين للشمال** (فلسفة ← تصميم ← لغة): بتفسّر **اللغة**.
- اقراها **من الشمال لليمين**: بتفسّر **الكود اللي قدامك**.

والسؤال اللي تسأله لأي لغة: **المصممين رفضوا يعملوا إيه، وليه؟** (What did its designers refuse to do, and why?)

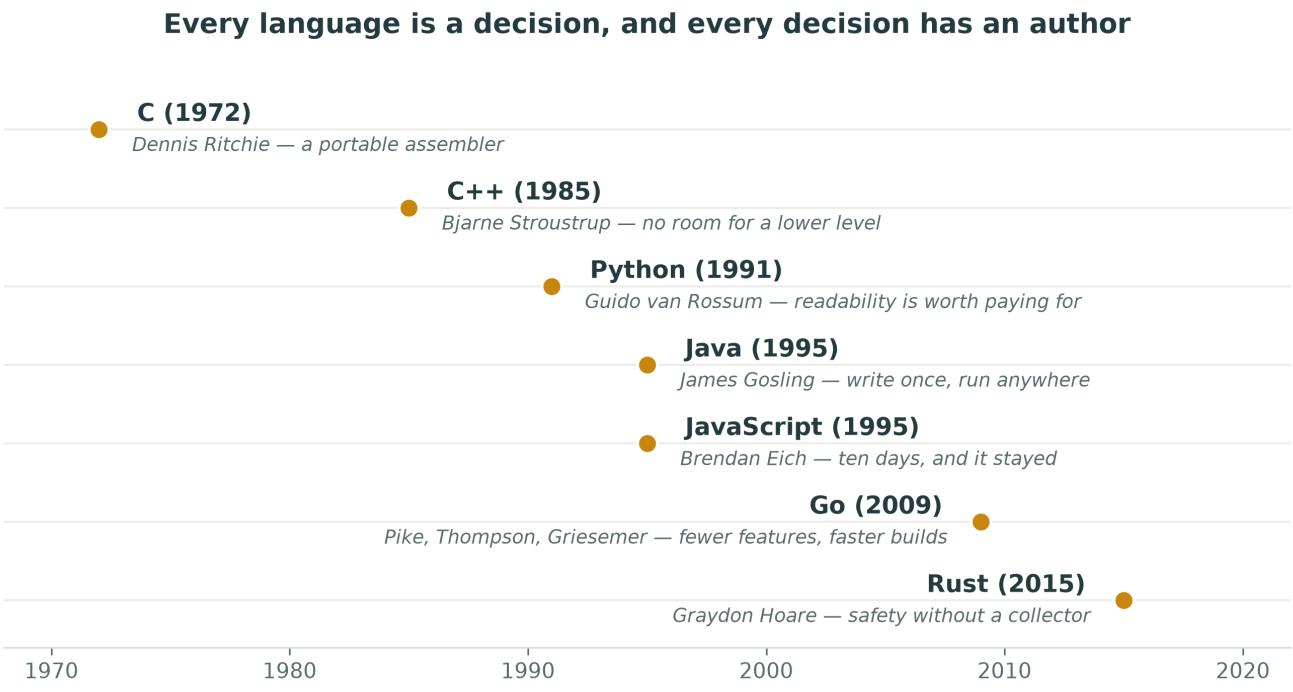
---

3) القرارات ليها أصحاب

جدول المصممين

Person	Language	القرار في سطر
Dennis Ritchie	**C**	Portable assembler. ثق في المبرمج.
Bjarne Stroustrup	**C++**	ماتسيبش مساحة لغة أقل مستوى منك.
Guido van Rossum	**Python**	القابلية للقراءة تستاهل إنك تدفع تمنها.
Graydon Hoare	**Rust**	أمان الذاكرة من غير garbage collector.
Pike, Thompson, Griesemer	**Go**	مميزات أقل، build أسرع، على نطاق كبير.

خمسين سنة قرارات (Timeline)



الرسم بيوضح إن كل لغة هي **إجابة لسؤال مختلف**، وللإجابة دي **مؤلف**:

اللغة	السنة	المؤلف	الفكرة
C	1972	Dennis Ritchie	portable assembler
C++	1985	Bjarne Stroustrup	no room for a lower level
Python	1991	Guido van Rossum	readability is worth paying for
Java	1995	James Gosling	write once, run anywhere
JavaScript	1995	Brendan Eich	عشر أيام، وفضلت
Go	2009	Pike, Thompson, Griesemer	fewer features, faster builds
Rust	2015	Graydon Hoare	safety without a collector

شرح فلسفة كل لغة

Python

- أولوية van Rossum: الكود **بيتقري أكتر بكتير ما بيتكتب**.
- النتايج في كل حتة: **الـ indentation مهمة** (مش ممكن تكتب Python indentation وحش، لأن الـ indentation الوحش ده **برنامج تاني خالص**)، **طريقة واحدة واضحة** للحاجة، و**keywords قليلة**.
- جرّب `import this` في Python prompt واقرا السطور اللي هتطلع (تسعتاشر سطر): ده **PEP 20 — The Zen of Python**، فلسفة اللغة بلسان مجتمعها.

C++

- قاعدة Stroustrup: C++ **ماينفعش تسيب فجوة تحتها** تجبرك تنزل لـ assembly أو C.
- فتطلع: **بتدفع تمن اللي بتستخدمه بس**، وعندك **تحكم مباشر في الذاكرة**، واللغة هتسمحلك تعمل **أي حاجة تقريبًا — حتى تدمّر نفسك**.
- نفس الفلسفة **بتطلّع الأداء والخطر مع بعض**.

Rust

- سؤال Hoare: هل أمان الذاكرة **لازم** garbage collector؟
- إجابة Rust: **ownership و borrowing** بيتشيّكوا وقت الـ compile — نقطة جديدة فعلًا في فضاء التصميم.
- عشان كده Rust طول سنين هي "**most admired**" في Stack Overflow survey.

Go

- **صغيرة عن قصد.** المصممين شالوا مميزات بتعتبرها لغات تانية أساسية.
- السبب: فريق كبير بيعمل compile لـ codebase كبيرة بيستفيد من **البساطة وسرعة الـ build** أكتر من **التعبيرية** (expressiveness).

> **نقطة مهمة:** أربع إجابات مختلفة، ومفيش واحدة غلط. دي إجابات على **أسئلة مختلفة**.

---

4) لغة C: منين جت وليه مهمة

التاريخ

- **Dennis Ritchie** في **Bell Labs** في أوائل السبعينات.
- اتبنت من لغة **B** بتاعة Ken Thompson، لغرض واحد: **تكتب Unix**.
- Unix اتكتب تاني بـ C سنة **1973** — أول نظام تشغيل كبير **مش مكتوب بـ assembly**.
- Ritchie و Thompson اتشاركوا **Turing Award سنة 1983**. Ritchie مات سنة 2011.

التاريخ ده بيفسّر شكل C

C ماتصممتش عشان تكون **لطيفة** أو **آمنة** أو **حديثة**. اتصممت عشان تكون **portable assembler**: حاجة تكتب فيها نظام تشغيل وبعدين **تنقله لجهاز تاني**. فاللغة:

- **صغيرة**.
- **قريبة من الـ hardware**.
- **بتثق فيك تمامًا**.

وكل حافة حادة في C هي **تمن القرار الواحد ده**.

C لسه تحتك

- **CPython** (الـ interpreter اللي بيشغّل Python بتاعك) مكتوب بـ **C**.
- كمان: **Linux**، نواة **Windows**، **SQLite**، **git**، **ffmpeg**.
- لما الـ `list` في Python بتكبر وبتضاعف مساحتها بهدوء — الحاجة اللي هتنفذها بإيدك في **Week 4** — التضاعيف ده بيحصل **في C**، كام طبقة تحت كودك.

الكتاب اللي لازم تملكه

> **Brian W. Kernighan and Dennis M. Ritchie**
> **The C Programming Language**, 2nd edition
> Prentice Hall, 1988 · ISBN 0-13-110362-8

- اسمه المشهور: **K&R**. خد **الطبعة التانية** — بتغطي **ANSI C**.
- الرابط: https://cs.princeton.edu/~bwk/cbook.html (صفحة Kernighan نفسه).
- الطبعة الأولى 1978، التانية 1988 هي اللي تشتريها. أقل من **300 صفحة**، ولسه **أحسن كتاب اتكتب عن اللغة** — جزئيًا لأن واحد من مؤلفيه **اللي اخترع اللغة**. مفيش مصدر أولي أقوى من كده.
- **تحذير:** ماتنزّلش النسخ الممسوحة (scans) اللي بتظهر في البحث، دي **منسوخة بشكل غير قانوني**.

النكتة اللي اتحققت

المحاضرة بدأت بـ "كل لغة تقدر تطبع `Hello, World`". ليه **البرنامج ده** بالذات في كل tutorial في كل لغة من 50 سنة؟ لأنه **أول برنامج في K&R**. كتاب واحد حدد سطر الافتتاح **للمهنة كلها**.

ليه C مهمة للكورس ده بالذات

C **بتخلّي الذاكرة مرئية**: `malloc`، `free`، **pointers**، `struct` — **مفيش حاجة مخفية ومفيش حاجة أوتوماتيك**. كل هيكل هنبنيه **شفاف في C ومخفي في Python**، وده بالظبط عيب Python الصريح اللي هنوصله في الجزء الخاص بـ Python. لو عايز تفهم **linked list** بجد، اكتب واحدة بـ C، ومش هتنسى يعني إيه pointer.

---

5) تصحيح من الدكتور (Knuth)

الدكتور على السبورة حط **Donald Knuth** جنب Stroustrup و van Rossum، وبيصلّح ده دلوقتي علنًا:

- Knuth **ماصمّمش لغة** هتكتب فيها production code.
- كتب **The Art of Computer Programming**، واخترع **TeX**، واخترع **literate programming**.
- موضوعه هو **الخوارزميات** — تحليلها وصحتها وتكلفتها.

> **نقطة مهمة:** Knuth **مش الجسر للغات، هو الجسر للكورس ده.**

ليه الدكتور صلّح قدام الكل؟

عشان ده **عادة عايزكم تقلدوها**: لما تعرف إن حاجة قلتها غلط، **تقول بوضوح، تصلّحها، وتكمّل. ماتمسحهاش بهدوء وتأمل إن محدش واخد باله.** في الهندسة، اللي **بيصحّح نفسه أوثق** من اللي "عمره ما بيغلط"، لأن التاني **مش موجود**.

الجملة المشهورة

> "Premature optimization is the root of all evil (or at least most of it) in programming." — Donald Knuth,
> **Structured Programming with go to Statements**, 1974

- الكلمة المهمة: **premature** (مبكّر).
- الناس بتقتبسها عشان تاخد **إذن إنها ماتفكرش في الأداء خالص**.
- المعنى الحقيقي في سياقها: **ماتعملش micro-optimise لـ 97% من الكود اللي مش مهم**، **عشان** تصرف مجهود حقيقي على الـ **3%** اللي مهمين.
- **إيجاد الـ 3% دي** هو شغل **تحليل التعقيد (complexity analysis)** — وده **Week 2**.

> **للامتحان:** الاقتباس مش معناه "الأداء مش مهم". معناه: **ماتحسّنش بدري، حدّد الجزء اللي فعلًا مهم الأول.**

---

6) البرادايمز: نفس الشغلانة بتلات طرق

المثال: مجموع مربعات الأعداد الزوجية في list

```python
# Procedural — describe the steps
total = 0
for n in numbers:
    if n % 2 == 0:
        total += n * n

# Functional — describe the transformation
total = sum(map(lambda n: n * n, filter(lambda n: n % 2 == 0, numbers)))

# Declarative (comprehension) — describe the result
total = sum(n * n for n in numbers if n % 2 == 0)
```

نفس النتيجة، نفس اللغة، **تلات طرق تفكير مختلفة**.

نفترض `numbers = [1, 2, 3, 4]` ونتتبع (trace)

- **الأعداد الزوجية:** 2 و 4.
- **مربعاتهم:** 4 و 16.
- **المجموع:** 20 — والتلات نسخ بتطلّع **20**.

وده شرح كل نسخة:

1. **Procedural:** بتقول **خطوة خطوة**: ابدأ بـ `total = 0`، لف على كل `n`، لو زوجي زوّد `n * n` . بتغيّر **state** (المتغير `total`).
2. **Functional:** `filter` بيختار الزوجي، `map` بيربّعهم، `sum` بيجمعهم. **بتركّب دوال** من غير ما تغيّر متغير.
3. **Declarative (comprehension):** بتقول **إيه** اللي عايزه (مجموع `n * n` لكل `n` زوجي) والنظام يعرف **إزاي**.

مصفوفة الـ paradigms

Most languages are multi-paradigm — so the choice of style is yours

	Procedural	Object-oriented	Functional	Declarative
C	core			
C++	core	core	supported	
Java	core	core	supported	
Python	core	core	supported	supported
JavaScript	core	supported	supported	
Rust	core	supported	supported	
Haskell			core	supported
SQL				core
Prolog				core

الـ paradigm مش قائمة مميزات

الـ Paradigm هو طريقة لتفكيك المشكلة، مش قائمة مميزات. الأربعة اللي هتقابلهم:

- **Procedural / imperative:** البرنامج **تسلسل خطوات بتغيّر الحالة (state)**. أمثلة: C، Pascal، **وجوه أي function في Python تقريبًا**.

- **Object-oriented:** البرنامج **objects بتملك بياناتها وبتعرّض عمليات عليها**. أمثلة: Java، C#، C++، و Python لما تكتب `class`.
  - ده بالظبط **أسلوب بناء فولدر** `dsa/`: الـ `Stack` **بيملك التخزين بتاعه** وبيعرّض `push` و `pop` و `peek`. **ماينفعش تدخل جوّاه، ولا المستخدمين بتوعه.**
- **Functional:** البرنامج **تركيب دوال**، وبتتجنب تغيير الحالة. أمثلة: Haskell، Elixir، Lisp، وفي Python: `map` و `filter` و closures.
- **Declarative / logic:** بتقول **إيه** عايزه والنظام يعرف **إزاي**. أمثلة: SQL، regular expressions، CSS، Prolog.

multi-paradigm و Python

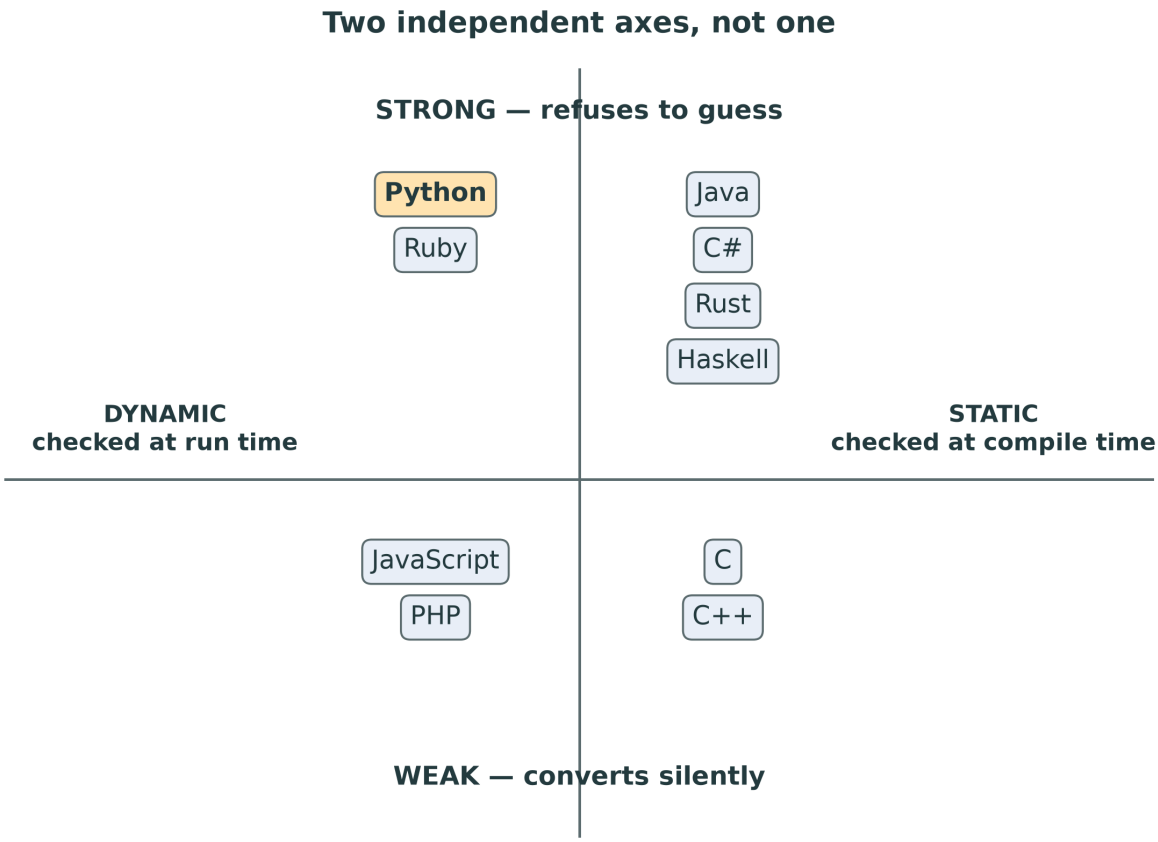
معظم اللغات **multi-paradigm**، و Python **بشدة**. ده **قوة وفخ**: Python هتسمحلك تكتب التلات أساليب، فـ **انت** اللي لازم تقرر أنهي أوضح.

> **للامتحان / نقطة مراجعة:** النسخة التالتة (comprehension) هي اللي الدكتور **كان يقبلها في الـ review**: **أقصر، بتقول اللي قصدته**، و**مابتبنيش list وسيطة** (intermediate list).

---

7) أنظمة الأنواع: محورين مش محور واحد

الطلبة بيدمجوا السؤالين دول في سؤال واحد. هما **مستقلين**.



المحور	بيجاوب على	الطرفين
Static vs Dynamic	**امتى** الأنواع بتتشيّك	static = وقت الـ compile، dynamic = وقت التشغيل (run time)
Strong vs Weak	**قد إيه بصرامة** الأنواع بتتفصل عن بعض	strong = **بيرفض يخمّن**، weak = **بيحوّل بصمت**

Python: dynamic بس strong

- **Dynamic:** الاسم ممكن يتربط بـ `int` دلوقتي وبـ `str` بعدين.
- **Strong:** بيطلّع error بدل ما يخمّن:

```
>>> "3" + 5
TypeError: can only concatenate str (not "int") to str
```

JavaScript: dynamic و weak

الـ JavaScript في نفس المثال بترجّع `"35"` (بتحوّل الرقم لنص بصمت).

نقطة مهمة: ولا سلوك فيهم bug. هما **قرارين مختلفين**: هل التحويل المفاجئ أنفع ولا الفشل الصريح؟ إجابة Python (**افشل بصوت عالي**) هي اللي **هتنقذك في الكورس ده**، لأن **تحويل نوع صامت جوه routine ترتيب** صعب جدًا تلاقيه.

مواقع اللغات على الرسم

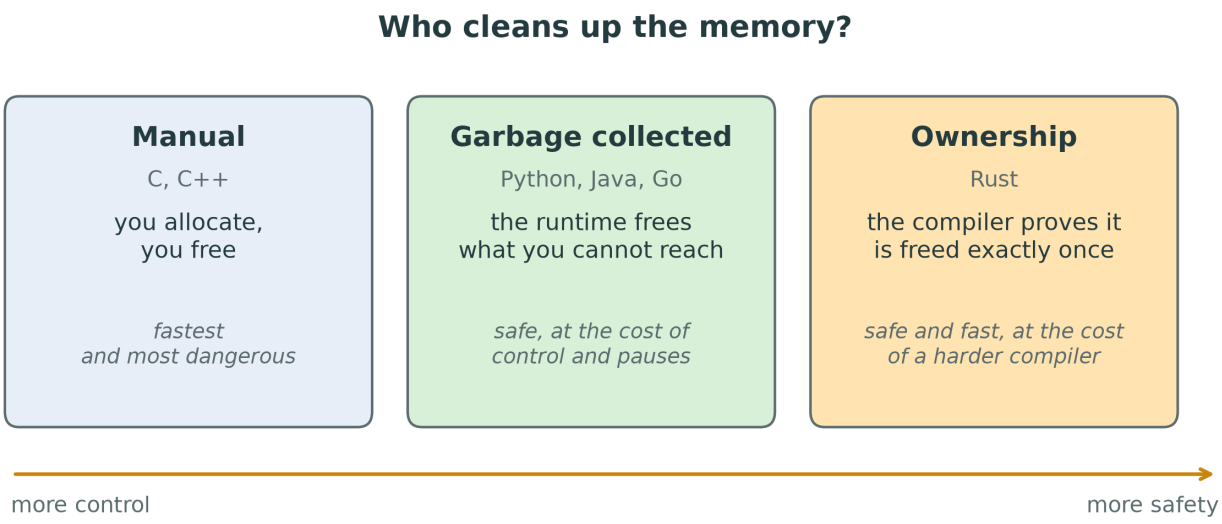
	Static (compile time)	Dynamic (run time)
Strong	Java، C#، Rust، Haskell	**Python** (فوق الشمال)، Ruby
Weak	C، C++	PHP، JavaScript

Type hints الاختيارية

- Python فيها **type hints اختيارية**: `def push(self, value: int) -> None:`
- **مش بتتنفذ وقت التشغيل** — الـ interpreter **بيتجاهلها**.
- لكن **الأدوات بتقراها، والبشر كمان**.
- الـ skeletons في `dsa/` **بتستخدمها**.
- اعتبرها **توثيق (documentation) محررك يقدر يتشيّك عليه**.

---

8) الذاكرة: مين بينضّف؟



النموذج	اللغات	الفكرة	التمن / الميزة
Manual	C, C++	إنت بتـ **allocate** وإنت بتـ **free**	**الأسرع**، ومصدر **عقود من الثغرات الأمنية**
Garbage collected	Python, Java, C#, Go	الـ runtime **بيحرر اللي مابقاش تقدر توصله**	**آمن**، على حساب **التحكم** وبعض **التوقفات (pauses)**
Ownership	Rust	الـ compiler **بيثبت وقت الـ build** إن الذاكرة بتتحرر **مرة واحدة بالظبط**	**آمن وسريع**، على حساب **compiler أصعب في إرضائه**

الرسم: **التحكم عند طرف، والأمان عند الطرف التاني.** C على الشمال (تحكم أكتر)، Python في النص وبتخبّي السؤال، Rust على اليمين (أمان أكتر).

ليه ده مهم لينا؟

لما تنفذ `dsa/linked_list.py` و"تشيل node ":

- في **C**: تنادي `free()`، وغلطة هنا = **crash أو ثغرة أمنية**.
- في **Python**: بتبطّل **تشاور** على الـ node والـ **garbage collector** هو اللي يتصرف.

المشكلة في الراحة دي

الراحة دي **بالظبط اللي بتخلّي الهيكل صعب إنك تشوفه**. هتبني linked list **من غير ما تلمس pointer واحد**. فاستخدم الـ **debugger**:

1. حط **breakpoint** في `dsa/linked_list.py`.
2. اضغط **F5**، واختار **pytest: current test file**.
3. شوف `node.next` وهو **بيتغيّر** في لوحة **Variables** وإنت بتتقدم بـ **F10**.

الإعداد في `README` بتاع الـ repository. **إنك تشوف reference بيتحرك أحسن من أي رسم**، حتى رسومات الدكتور.

---

9) نقارن اللغات بأمانة

إنت بتسأل عن إيه بالظبط؟

"أنهي لغة أحسن؟" مش سؤال. دي أسئلة:

- أحسن **لأنهي مهمة**؟ (web service، embedded firmware، data analysis، لعبة)
- أحسن **لمين**؟ (إنت لوحدك بتتعلم؟ ولا 40 مهندس لمدة 10 سنين؟)
- أحسن بأنهي **مقياس**؟ (سرعة الكتابة، سرعة التشغيل، سوق الشغل، الأمان)
- أحسن **امتى**؟ (المشروع ده؟ ولا اللي هتصيّنه لحد 2035؟)

غيّر **أي واحد** من دول والإجابة هتتغيّر. لغة **مثالية لسكريبت تحليل بيانات في ويك إند** اختيار وحش لـ **firmware على جهاز فيه 64 KB RAM**، والعكس صحيح. ده **مش دبلوماسية، ده هندسة.**

صورة المطرقة

الدكتور رسم **مطرقة محاطة بمسامير معووجة**. المطرقة **أداة ممتازة**، لكن المسامير معووجة **لأن المطرقة كانت الأداة الوحيدة اللي الشخص يملكها**.

نقطة مهمة: اتعلّم أكتر من لغة — **مش عشان اللغة التانية نفسها**، لكن لأنك **ماتقدرش تشوف القرارات اللي لغتك الأولى عملتها** إلا لما تشوف مجموعة قرارات مختلفة.

الـ rankings بتختلف — وده هو الدرس

Index	بيعدّ إيه فعلًا	بيتوّج مين
TIOBE	عدد نتائج محرك البحث لعبارة "X programming"	Python
Stack Overflow Survey	الاستخدام اللي المشاركين بيبلّغوا عنه	JavaScript
GitHub / Octoverse	repositories و commits	TypeScript / Python
PYPL	بحث Google عن "X tutorial"	Python

أدوات قياس مختلفة، إجابات مختلفة. **ولا واحد فيهم بيقيس الجودة.**

أرقام TIOBE — September 2026

اللغة	النسبة
Python	17.76%
C	10.28%
C++	8.67%
Java	7.54%
C#	4.22%
JavaScript	2.76%
Visual Basic	2.55%

- **Rust** في المركز **10**.
- TIOBE بيعدّ **نتايج محركات البحث**، فهو **مؤشر على قد إيه الناس بتكتب وتسأل** عن اللغة — **مش** قد إيه كود شغال في production، **ولا بالتأكيد** جودة اللغة.
- **Visual Basic فوق Rust** المفروض يقولك كل حاجة عن اللي المؤشر ده هو وملوش.

مقارنة مع Stack Overflow

- **JavaScript** هي الأولى في الاستخدام المُبلّغ عنه تقريبًا **كل سنة من 2011**.
- **Rust** هي **most admired**: حوالي **72%** من المطورين اللي بيستخدموها **عايزين يكملوا**.
- **الاستخدام والإعجاب سؤالين مختلفين**، بيتقاسوا بطرق مختلفة، وإجاباتهم مختلفة.

تعمل إيه بالـ rankings؟ حاجتين بس

1. استخدمها عشان تشوف **اتجاهات (trends) على سنين**، **مش ترتيب شهر واحد**. الحاجة المهمة مش إن Python الأولى في سبتمبر — لكن إنها **طلعت لهناك وفضلت**.
2. استخدمها عشان تتأكد إن اللغة **مش بتموت**، لأن لغة من غير مجتمع = **مفيش مكتبات، ومفيش إجابات، ومفيش شغل**.

> **تحذير: ماتستخدمهاش أبدًا لاختيار لغة لمهمة.** ده بتقراه عن **المهمة** نفسها.

> **تحذير:** كل الأرقام دي تاريخها **September 2026** وهتبقى غلط قريب. **أي slide بتذكر ترتيب من غير تاريخ بتكذب عليك بالحذف** (lying by omission).

الـ ecosystem بيغلب الـ syntax

إنت لما بتتبنى لغة، بتتبنى إيه فعلًا:

- **Libraries:** حد حل مشكلتك قبل كده **كويس**؟
- **Tooling:** debugger، profiler، package manager، formatter، test runner.
- **Community:** سؤالك **اتجاوب** قبل كده؟
- **Interoperability:** تقدر **تنادي الحاجة السريعة** المكتوبة بلغة تانية؟
- **Longevity:** هتبقى **متصانة بعد 10 سنين**؟

الـ syntax هو الجزء اللي **بتبطل تلاحظه بعد أسبوعين**. الـ ecosystem هو الجزء اللي **بتعيش جواه سنين**. لما فريق بيتخانق على اللغة، **القائمة دي** هي اللي بيتخانقوا عليها فعلًا، حتى لو فاكرين إنهم بيتخانقوا على الـ semicolons.

---

10) ليه Python في الكورس ده

الحجة المؤيدة (The honest case for)

- **الميدان رايح فين:** research، scientific computing، data analysis، و**تقريبًا كل أدوات AI/ML/DL**.
- **Bindings:** الشغل العددي التقيل في الحقيقة **C أو C++ أو CUDA أو Fortran** من تحت، و Python هي **الطبقة اللي بتتحكم منها**. **NumPy مش Python بطيء — هي C سريعة بـ "مقبض Python".**
- **تكلفة القراءة:** Python **قريبة كفاية من الـ pseudocode** فالخوارزمية **بتتنقل على الصفحة من غير ما تتشوّه**. ده مهم في كورس **الخوارزمية فيه هي المحتوى**.
- **هتقابلها تاني:** في **كل برامجك** — AI و Medical Informatics و Software Engineering.

نقطة الـ **bindings** هي السهم اللي الدكتور رسمه من "سحابة "Python لصندوق الـ production، وهي سبب إن Python في المكانة اللي هي فيها. لغة **سهلة الكتابة بس بطيئة التشغيل** هتبقى لعبة (toy). لغة **سهلة الكتابة وتقدر تنادي الحاجة السريعة** هي **بنية تحتية** (infrastructure).

الحجة المضادة — بالنسبة لنا

Python **فيها بالفعل كـ built-ins** معظم اللي الكورس بيدرسه:

هتبني	Python عندها بالفعل
Dynamic array	`list`
Hash map	`dict`
Stack / queue	`list` ، `collections.deque`
Sorting	Timsort — `list.sort()` ، `sorted()`
Priority queue	`heapq`

نقطة مهمة: الأداة بتخبّي بالظبط اللي إحنا هنا عشان نبصّ عليه.

القاعدة اللي في `dsa/__init__.py`

> No module may use the built-in it is reimplementing.

يعني: **ممنوع أي module يستخدم الـ built-in اللي بيعيد تنفيذه.**

- `dsa/dynamic_array.py` **ممنوع يكون مبني على** `list` — بيستخدم `ctypes` ، والدالة `make_block()` **معطاة لك جاهزة** لأن تخصيص ذاكرة خام **مش هو الدرس**.
- `dsa/hashmap.py` **ممنوع يكون مبني على** `dict` .
- لو لفّيت حوالين القاعدة، **الاختبارات ممكن تنجح** وإنت **ماتعلمتش حاجة**. **الاختبارات مش هي الهدف، هي الدليل على الهدف.**

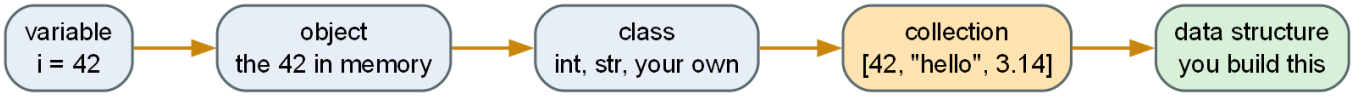
Python هي الوسيلة مش الموضوع

Python هي vehicle الكورس، مش subject بتاعه. كل اللي هتبنيه — تكلفة الإدراج (insertion)، ليه hash map بتتدهور، ليه الـ pivot مهم في quicksort — **بينتقل بدون تغيير** لـ C++ و Java و Rust و Go. الشيء الوحيد اللي **مابينتقلش** هو الـ syntax، **والـ syntax هو الجزء الرخيص**.

11) من المتغير لهيكل البيانات

السلّم (The ladder)

variable → object → class → collection → data structure




```
i = 42                    # a name bound to an object
s = "hello"               # an object of class str
xs = [42, "hello", 3.14]  # a collection of references
```

خمس درجات. الكورس عايش على **الدرجة الأخيرة**، لكن مالوش معنى من غير الأربعة اللي قبلها يبقوا **ثابتين**.

نقطة مهمة: في Python المتغير **مش صندوق فيه قيمة**. هو **اسم مربوط بـ object**.

الفرق بين C و Python

- في **C**: `int i = 42;` بيحجز **4 bytes** وبيحط الرقم فيهم. **الاسم هو التخزين نفسه.**
- في **Python**: `i = 42` بيعمل (أو يلاقي) **integer object** في مكان في الذاكرة وبيربط الاسم `i` بيه. **الاسم label مش container.**

نتيجة 1: الـ aliasing

```
a = [1, 2, 3]
b = a            # b is not a copy. Both names point at ONE list.
b.append(4)
print(a)         # [1, 2, 3, 4]
```

Trace:

1. `a = [1, 2, 3]` ← بيتعمل list واحدة و `a` بيشاور عليها.
2. `b = a` ← **مفيش نسخة**. `b` بيشاور على **نفس الـ list**.
3. `b.append(4)` ← بنعدّل **الـ list الوحيدة**.
4. `print(a)` ← `a` بيشوف التعديل لأنها **نفس الـ list**: `[1, 2, 3, 4]`.

تحذير: ده فخ هتقع فيه الترم ده: `b = a` **مش بيعمل copy.** كتير من الأخطاء في linked lists و arrays مصدرها ده.

نتيجة 2: الـ list بتخزّن references

```
xs = [42, "hello", 3.14]
```

الـ `list` في Python **ماتخزّنش الـ objects**، بتخزّن **references** ليها، وده **سبب إن list واحدة تشيل 3 أنواع مختلفة**. لما هتبني `dsa/dynamic_array.py` على block خام من `ctypes`، **هتخزّن references برضه** — وهتشوف **الماكينة اللي `list` بتخبّيها عادةً**.

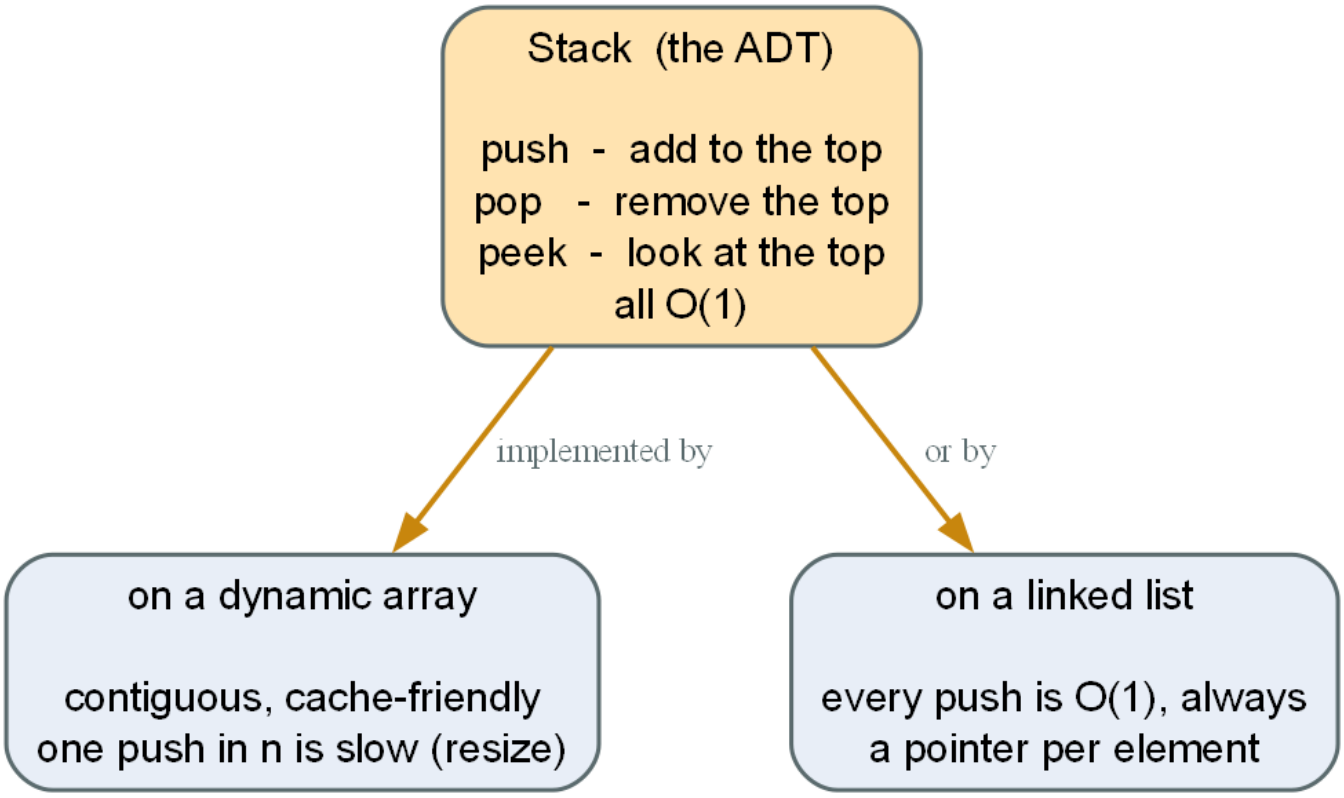
ADT مقابل الـ implementation

Abstract Data Type (ADT) — النوع المجرد للبيانات

: **العقد (contract)**: إيه العمليات الموجودة، **معناها** إيه، **وتكلفتها** كام. **مفيش أي حاجة** عن إزاي البيانات متخزّنة.

Data structure — هيكل البيانات

: **ترتيب فعلي للبيانات في الذاكرة** بيحقق العقد ده.



ADT واحد، هياكل كتير. الـ ADT اسمه **Stack** بيقول بس:

- `push` بيضيف.
- `pop` بيشيل **آخر** عنصر اتضاف.
- `peek` بيبص عليه.
- **كلهم O(1).**

العقد ده ممكن يتنفذ بـ **dynamic array** أو بـ **linked list**. **الاتنين stacks حقيقية.**

	Stack على dynamic array	Stack على linked list
التخزين	**متجاور (contiguous)**، **cache-friendly**	**pointer لكل عنصر**
الـ push	**واحدة من كل n بطيئة** (resize)	**كل push هي O(1) دايمًا**

بيختلفوا في: **سلوك الـ cache**، **الـ overhead في الذاكرة لكل عنصر**، وهل **عملية واحدة ممكن تبقى بطيئة أحيانًا**.

> **للامتحان: الاختيار بينهم هو الهندسة**، وماتقدرش تختار من غير ما تعرف **تكلفة كل واحد** — عشان كده **التعقيد (complexity) بييجي قبل الهياكل**، في Week 2.

قاموس المصطلحات

الدكتور هيستخدم **اللغتين** طول الترم:

English	العربية
Data structure	هيكل البيانات
Algorithm	الخوارزمية
Abstract Data Type	النوع المجرد للبيانات
Complexity	التعقيد
Array	المصفوفة
Linked list	القائمة المتصلة
Stack	المكدس
Queue	الطابور
Tree	الشجرة
Graph	الرسم البياني
Hash table	جدول التجزئة
Sorting	الترتيب
Searching	البحث
Recursion	الاستدعاء الذاتي

Know-How

تلات حاجات مختلفة، **والتالتة بس هي اللي ليها قيمة كبيرة**:

1. **Know-what** — تقدر **تعرّف** stack. **ساعة قراءة.**
2. **Know-how** — تقدر **تبني واحد شغّال**. **تمارين الكورس.**

3. **know-why و Know-when** — تقدر تقول **ليه stack مش queue**، وبكام. **امتحانات الكورس، وشغلك بعدين.**

Know-How هي الكلمة اللي الدكتور كتبها على السبورة ودوّر عليها، وهي **عمود الكورس الفقري**.

- السبب إن `dsa/` عبارة عن **skeletons كلها بترمي** `NotImplementedError` هو **المستوى 2**.
- السبب إن الامتحان هيسألك **أنهي هيكل وليه** مش **"عرّف الـ "heap** هو **المستوى 3**.

مثال المستويات:

- طالب **حفظ** إن الـ hash map هي = "O(1) average" **مستوى 1**.
- طالب **يقدر ينفذ واحدة** = **مستوى 2**.
- طالب **يقدر يفسّر ليه الـ hash map بتاعته اتدهورت لـ O(n) على بيانات حقيقية، وبعدين يصلّحها** = **مستوى 3** — وده **اللي يتوظّف**.

> **للامتحان:** توقّع أسئلة "**اختار الهيكل المناسب وبرّر بالتكلفة**" أكتر من أسئلة التعريف.

---

12) التكلفة هي الموضوع كله

الخوارزمية إيه؟

الخوارزمية (Algorithm) = **إجراء محدود (finite) وغير غامض (unambiguous)** بيحوّل input لـ output **صح**.

بنهتم بتلات خصائص، **بالترتيب ده**:

1. **Correctness (الصحة):** بتطلّع الإجابة **الصح، دايمًا**؟
2. **Cost in time:** الشغل **بيكبر إزاي** لما الـ input يكبر؟
3. **Cost in space:** الذاكرة **بتكبر إزاي** لما الـ input يكبر؟

الصحة الأول. إجابة **سريعة وغلط** ملهاش قيمة، و"**اشتغلت على 3 test cases عندي**" مش صحة. عشان كده **تمارين الكورس بتتسلّم كـ tests**: فولدر `tests/` بيعرّف يعني إيه "صح" **بشكل تقدر الماكينة تتشيّك عليه**، **قبل** ما تكتب سطر من التنفيذ.

قيس، ماتدّعيش (Measure, do not assert)

ماتصدّقش الدكتور إن binary search أحسن من linear search. **قيسها.**

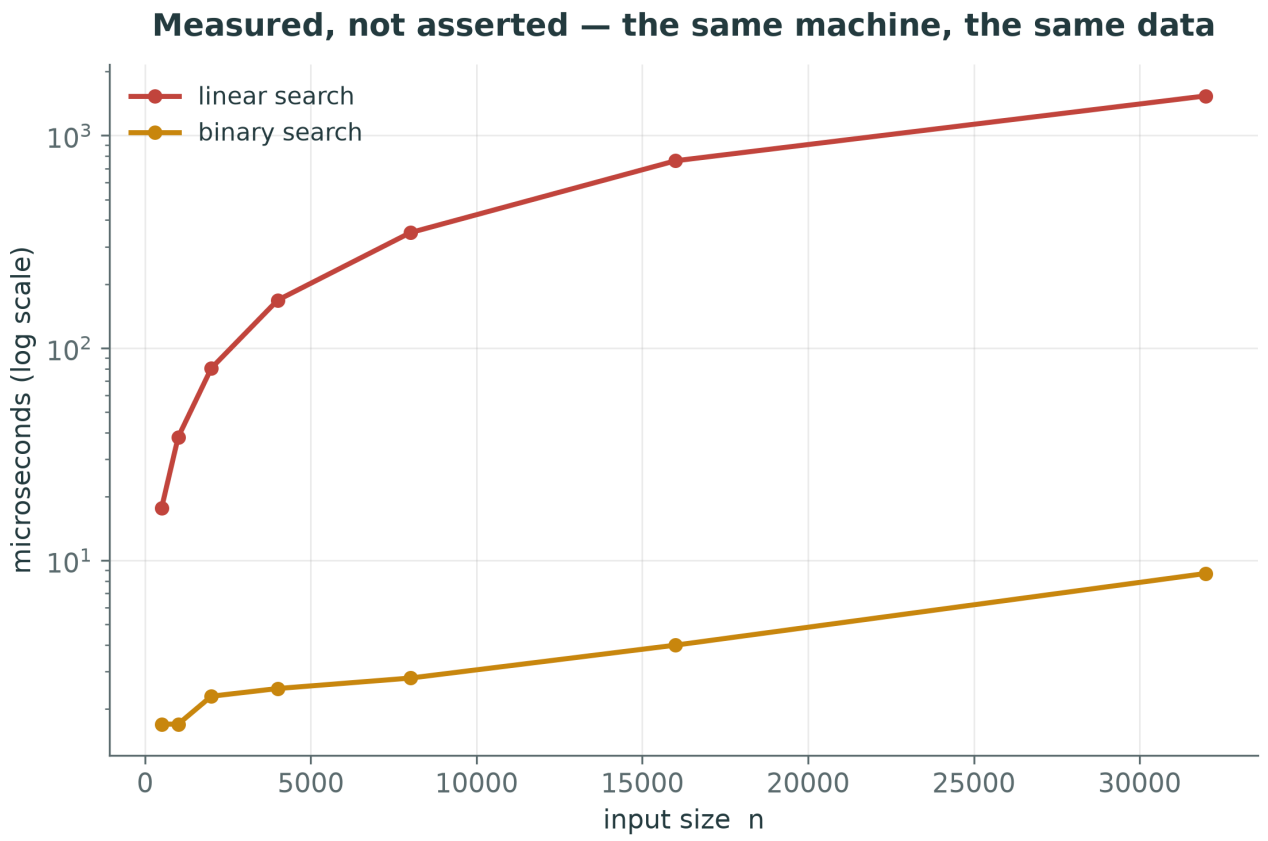
```
from viz.complexity import measure, plot_growth
import random

sizes = [1000, 2000, 4000, 8000, 16000]
make = lambda n: (sorted(random.random() for _ in range(n)), 0.5)

plot_growth(
    {"linear": measure(lambda a: linear_search(*a), sizes, make),
     "binary": measure(lambda a: binary_search(*a), sizes, make)},
    reference=["n", "log n"],
)
```

- `sizes` : أحجام الـ input اللي هنجرّب عليها.
- `make(n)` : بتعمل list **مرتبة** من `n` رقم عشوائي، ومعاها القيمة `0.5` اللي هندوّر عليها.
- `measure(...)` : **بتقيس زمن** الدالة على كل حجم وبترجّع **أحسن زمن** عند كل حجم.
- `plot_growth(...)` : بترسم **قياساتك** مقابل **منحنيات مرجعية** (`n` و `log n`).

> **نقطة مهمة:** `viz/complexity.py` **مكتوب جاهز** — هو **infrastructure** مش تمرين.



الرسم: **توقيتات حقيقية على جهاز واحد**. linear search **بتطلع**، و binary search **بالكاد بتتحرك** من **500** عنصر لـ **32,000**. (الـ y بمقياس **log**.)

نقطة التقاطع (crossover)

الرسم بيدّيك حاجة جدول Big-O مابيدّيهاش: **نقطة التقاطع**. linear search **غالبًا بتكسب على الـ inputs الصغيرة**، لأن binary search **بتعمل شغل أكتر لكل خطوة** فالـ **constant factors** بتسيطر. **Big-O بترمي الثوابت دي عن قصد** — وده اللي بيخلّيها **مفيدة على نطاق كبير ومضلّلة عند n = 20**.

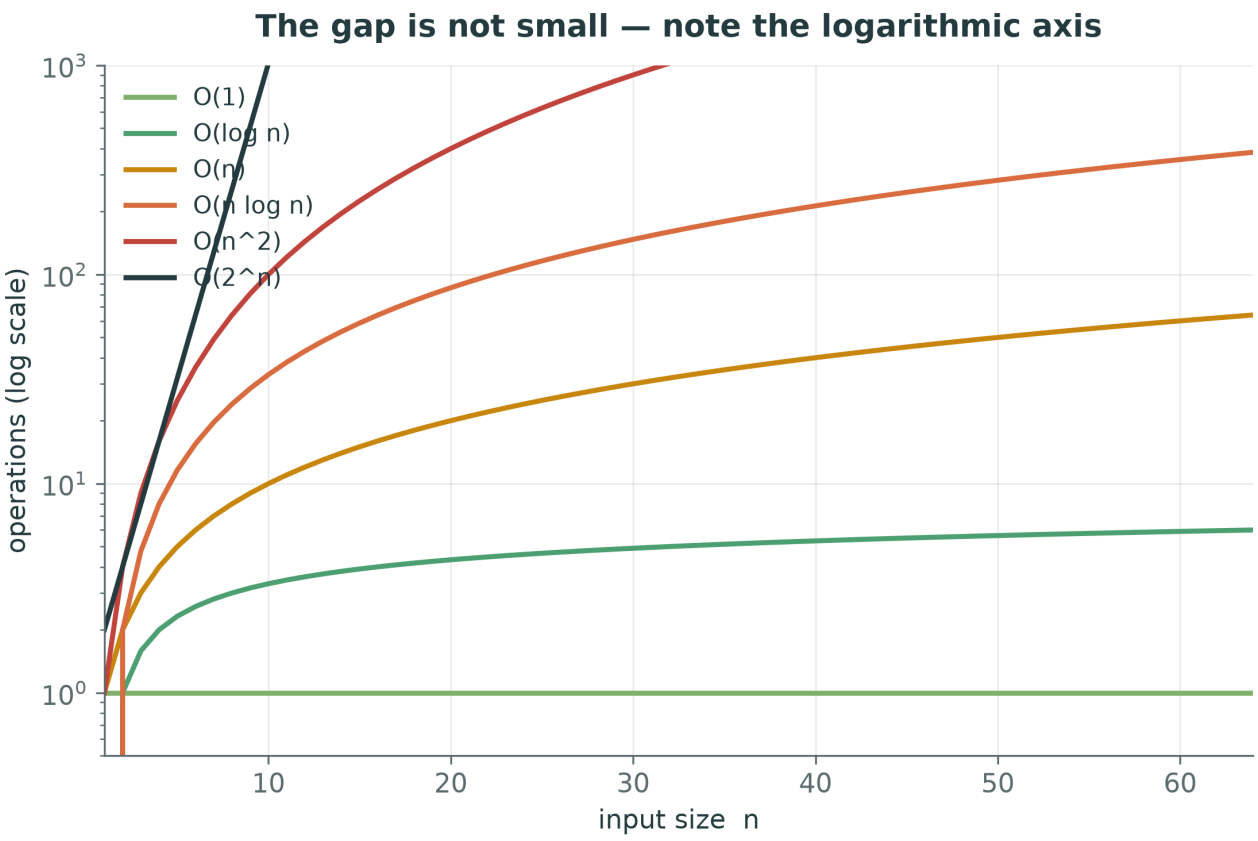
> **نقطة مهمة:** العادة اللي الدكتور عايزها: لما مش متأكد من التكلفة، **ماتتجادلش وماتخمّنش**. **قيس، ارسم، وبص.**

الفجوة مش صغيرة

$O(2^n)$	$O(n^2)$	$O(n \log n)$	$O(n)$	$O(\log n)$	n
1,024	100	33	10	3	10
$\sim 1.3 \times 10^{30}$	10,000	664	100	7	100
—	1,000,000	9,966	1,000	10	1,000
—	10^{12}	$\sim 2 \times 10^{7}$	10^{6}	20	1,000,000

الأرقام جت منين؟ (للفهم)

- **log n** هنا هو log بالأساس 2 (كم مرة تقسم على 2 لحد ما توصل لـ 1). مثلًا **$\log_2 10 \approx 3.3$** فبيتقرّب لـ **3**، و **$\log_2$ 1,000,000 ≈ 19.9** فبيتقرّب لـ **20**.
- **n log n**: `10 × 3.3 ≈ 33` و `100 × 6.6 ≈ 664` و `1000 × 9.97 ≈ 9,966`.
- **n^2**: `$10^2$ = 100` و `$1000^2$ = 1,000,000` و `$1{,}000{,}000^2 = 10^{12}$`.
- **2^n**: `$2^{10}$ = 1,024` و `$2^{100} \approx 1.3 \times 10^{30}$`. الشرطة "—" معناها إن الرقم **ضخم لدرجة إنه ملوش لازمة عمليًا**.



نفس الستة منحنيات مرسومة. **لاحظ المحور الرأسي logarithmic** — من غيره، كل حاجة ماعدا 2^n كانت هتبقى **مسطّحة على الأرض**.

عند n = 1,000,000:

O(log n) = 20 خطوة، O(n²) = تريليون خطوة.

ليه ده مهم

اقرا **آخر صف** لحد ما يبقى مش مريح. ده **مش فرق أكاديمي**، ومش حاجة **الجهاز الأسرع بيصلّحها**:

- لو خوارزميتك **O(n²)** والبيانات **كبرت 10 مرات**، الوقت **بيكبر 100 مرة**.
- لو اشتريت **جهاز أسرع مرتين**، رجّعت **عامل 2 بس**.

اختيار الهيكل الصح هو طريقك **تنتقل بين صفوف الجدول ده. ده الكورس كله في جدول واحد.**

للامتحان: احفظ الترتيب: **O(1) < O(log n) < O(n) < O(n log n) < O(n²) < O(2ⁿ)**، وافهم **ليه** الفرق يبقى ضخم مع كبر n.

13) الكورس نفسه رسميًا

تلات برامج، تلات لوائح، كورس واحد

Software Engineering	Bio / Medical Informatics	AI	
IS122	**IS122**	**CS2101**	Code
3 (2 lec + 2 lab)	3 (2 lec + 2 lab)	3 (2 lec + 2 lab)	Hours
CS012, MATH012	CS012, MATH012	CS1002 OOP	Prerequisite
Sophomore, Sem. 1	Level 2	Level 2, Sem. 3	Level
2013, p. 38	2014, p. 35	2020, p. 44	Bylaw

أكواد مختلفة، **نفس الموضوع، نفس الساعات، نفس القاعة**.

- الاسم العربي بيختلف شوية: **هياكل البيانات و الخوارزميات** في لائحة الـ AI، و**هياكل البيانات وتحليل الخوارزميات** في الاتنين التانيين.
- لوائح الـ PDF الأربعة موجودة في الـ repository في `docs/course/regulations/`، **وكل ادّعاء مرفق برقم الصفحة** عشان تتأكد بنفسك.

اللائحة بتقول الكورس فيه إيه

نص لائحتي **2013 و 2014 متطابق حرفيًا**:

> "Introduce the fundamental concepts of data structures and the algorithms that proceed from them. Topics include **recursion**, the underlying philosophy of **object-oriented programming**, fundamental data structures (**stacks, queues, linked lists, hash tables, trees, and graphs**) and the basics of **algorithmic analysis**, an introduction to the **principles of language translation**."

- لائحة **AI 2020** بتزوّد بالاسم: **arrays، heaps، priority queues، sorting، searching، graph searches، tree traversals**.
- **خطة الـ 15 أسبوع** بتغطّي **الاتحاد** بين الاتنين، و `docs/course/02-coverage.md` بيربط **كل موضوع معلن** بالـ module والـ test اللي بيقيّمه.

حاجتين مهمين عن مكان الكورس

1) الكورس بيفترض أقل مما تتخيل. اللائحة بتقول إن الكورس **بيعلّم** recursion وفلسفة الـ OOP، **مش بيفترضهم**. لو إنت في Bio، **مفيش حاجة قبل الكورس ده علّمتك إياهم**، وده **متوقع**، وعشان كده **Week 3 هو recursion**.

2) بالنسبة لبرنامجين من التلاتة، ده هو الكورس الخوارزميات الوحيد اللي هتاخده في حياتك.

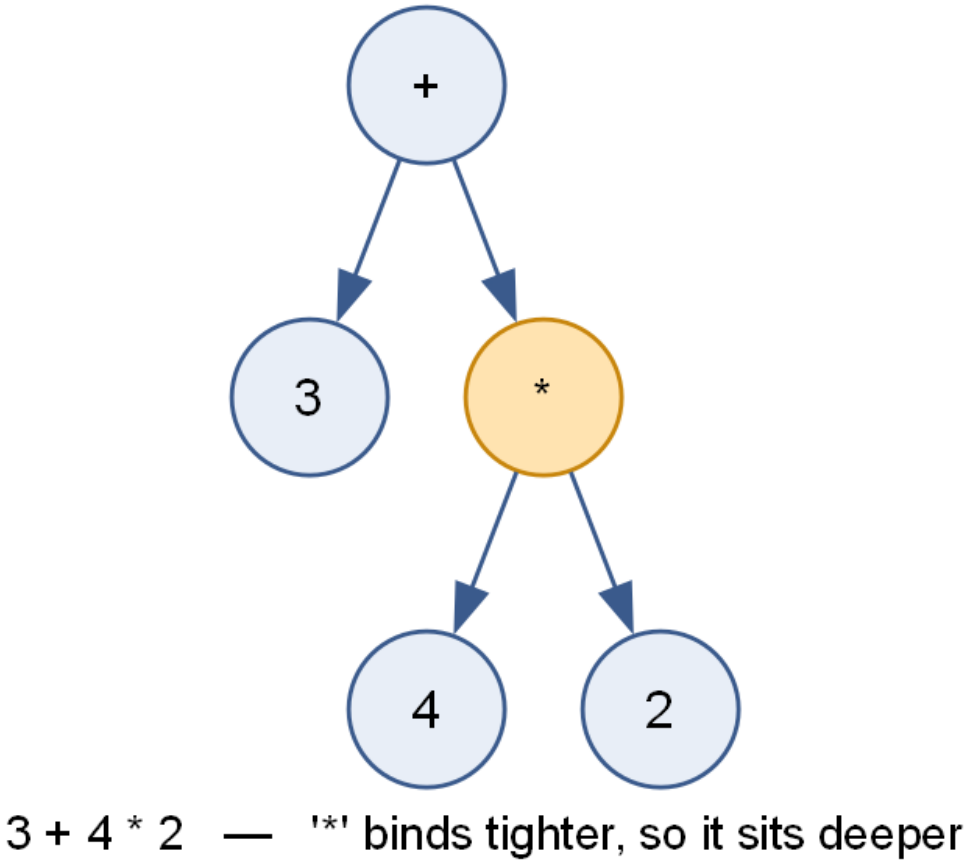
- لائحة AI فيها كورس تابع: **AI3001 — Analysis and Design of AI Algorithms**.
- لائحتي **2013 و 2014 مفيهمش ولا كورس تابع**.

- يعني لو إنت **SWE أو Bio**، اللي ماتتعلمتوش هنا **مش هتتعلمه في أي مكان تاني**.

الكورس ده بيفتح إيه

Database Systems و **Computer Networks** و **Computer Vision** كلهم **بياخدوا الكورس ده كـ prerequisite** — ولـ SWE كمان **SWE141 Software Construction**، وهو **المكان اللي "principles of language translation" بتكبر فيه لـ parsers و grammars**.

في **Week 15** هتبني **النسخة الصغيرة**: **نص يدخل، شجرة تطلع، وإجابة في الآخر.**



الرسم: `3 + 4 * 2` **بعد الـ parsing**.

- `*` **بيربط أقوى** من `+` فبيقعد **أعمق في الشجرة** وبيتحسب **الأول**.
- العملية `4 * 2 = 8` بتتحسب في الأول، وبعدين `3 + 8 = 11` .
- **الأولوية (precedence) مش جدول أرقام، هي شكل (shape).**

من السنة الجاية

لائحة الكلية 2026 بتستبدل التلاتة بكورس واحد: **CS201**، **3 credit hours**، prerequisite **CS101**، **Sophomore Fall**، ومادة **إجبارية في تخصص كل البرامج السبعة**. **ده مش بيحكمك** (مش لائحتك)، موجود في الـ repository لو عايز تشوف الكلية رايحة فين.

إزاي تنجح (How you pass)

نفس التلات قواعد في **كل اللوائح التلاتة**:

- **الامتحان النهائي التحريري 60% = (final)**، والـ coursework الـ **40%** الباقيين.
- النجاح محتاج **60% ≥ إجمالي و 30% ≥ من الـ final** — **الاتنين مع بعض**.
- **75% ≥ حضور**، وإلا إنت **محروم** وماتقدرش تدخل الـ final أصلًا.

المصادر: SWE 2013 صفحات 13–11 و 18؛ Medical Informatics 2014 صفحات 13–11 و 18؛ AI 2020 صفحات 16–14 و 19.

توزيع الـ 40%

اللوائح التلاتة بتختلف **بس** في تقسيم الـ 40%:

- **SWE و Bio**: الـ midterm لازم **على الأقل 20%**.
- **AI**: الـ midterm **على الأقل 10%**.
- **ولا واحدة** بتسمح إن أي مكوّن لوحده **يعدّي 60%**.
- **التقسيم الدقيق لكورسنا** هيتعلن في المحاضرة وعلى قناة WhatsApp، وهيتكتب في `docs/course/00-course-guide.md` لما يتثبّت. **لحد دلوقتي اعتبره لسه ماتقرّرش، والتلات قواعد فوق مُلزمة.**

> **للامتحان / تنبيه:** الدكتور بيأكد على قاعدة الـ "30%: الطلبة بيخسروا سنين بسببها"، ومثاله: اللي يجمّع **35 من 40** في الـ coursework ويجيب **25%** في الـ final **راسب** بغض النظر عن المجموع. **الـ coursework ماتقدرش تنقذ final تحت 30%.**

> **ملحوظة مهمة:** (ده حساب مني على أساس التقسيم 60/40 اللي الدكتور ذكره، مش من المحاضرة.) في المثال ده المجموع كمان **تحت 60 أصلًا**: 25% من 60 = (15 + 35) **50**. وبشكل عام لو الـ coursework **كاملة (40)** فلازم تجيب **20 من 60 على الأقل (حوالي 33%)** في الـ final عشان توصل لـ 60 إجمالي. يعني **عمليًا أنت محتاج final قوي** بغض النظر عن الأرقام.

الحضور = حساب مش تهديد

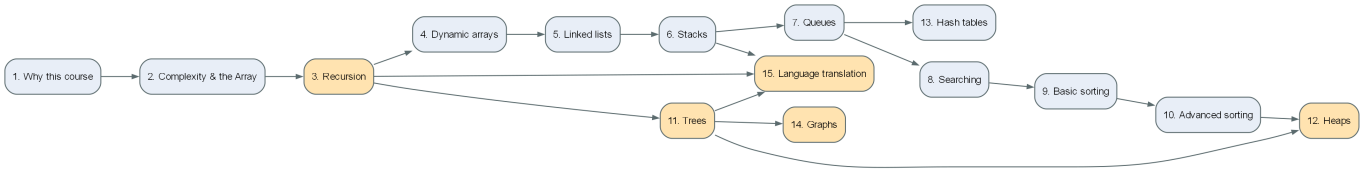
75% من كورس **15 أسبوع** معناها إنك تقدر **تغيب حوالي 3 جلسات تقريبًا — مش 5.**

سلّم التقديرات (Grade scale)

- من **A+ (≥ 97%)** لـ **F (< 60%)**.
- **D** هو **أقل نجاح** في أي كورس.
- اللوائح **بتختلف** هل A+ بـ **4.00** ولا **4.33** نقطة؛ **3 من الـ 4 مستندات** بتقول **4.00**.
- الخلاف ده، واتنين غيره، **موثّقين مش مخبّيين** في `docs/course/regulations/dsa-in-your-program.md`.

إزاي هنشتغل (How we will work)

- **Lecture:** الفكرة، التكلفة، **وليه موجودة**.
- **Lab: إنت اللي بتنفذ**. الـ skeletons في `dsa/` **كلها بترمي** `NotImplementedError` **من أول يوم**. **ده هو الـ assignment.**
- **Tests:** فولدر `tests/` بيعرّف "صح". الأمر `pytest -m challenge` **بيشغّل كل التمارين**.
- **Visualisation:** فولدر `viz/` **بيرسم** arrays و lists و trees و graphs، **وبيحرّك** الـ sorts بتاعتك، **وبيرسم** التعقيد اللي قسته. **عمره ما بيعمل import لـ** `dsa/` فبيشتغل مع **أي حاجة تكتبها**.



خريطة الكورس: **15 أسبوع**. **السهم معناه إن الأسبوع اللي بعده فعلًا محتاج اللي قبله** — عشان كده:

- **(3) recursion قبل (11) trees**.
- **trees قبل الـ (15) parser**.
- (13) Hash tables بتيجي بعد (7) Queues.
- (12) Heaps بتعتمد على (11) Trees و (10) Advanced sorting.

الخطة الكاملة (الـ module وملف الـ test لكل أسبوع) في `docs/course/01-study-plan.md`، والخريطة اتولّدت منها.

الكورس **كله بيفشل في أول يوم** — **ده هو المقصود**، وهي **النسخة الأمينة من الكورس**: **مش مطلوب منك تعيد حل الدكتور، مطلوب منك تحقّق مواصفة (specification).**

---

14) الملخص: ست حاجات تحفظها

1. **اللغات مش زي بعض.** كل لغة **مجموعة قرارات** ليها **صاحب وسبب**.
2. **"أنهي أحسن؟" مالهاش إجابة** لحد ما تقول **أحسن لإيه، ولمين، وبأنهي مقياس**.
3. مؤشرات الشعبية **بتقيس حاجات مختلفة** وبتختلف. **ولا واحد بيقيس الجودة. راجع التاريخ دايمًا.**
4. بنستخدم **Python** لأنها **بتتقري زي الخوارزمية** — وبنعيد **تنفيذ الـ built-ins بتاعتها** لأنها غير كده **بتخبّي الموضوع**.
5. **الـ ADT عقد**، و**الـ data structure طريقة لتنفيذه. الاختيار بينهم هو الهندسة.**
6. **Know-how أحسن من know-what؛ know-why أحسن من الاتنين.**